

AI INVESTIGATION ENGINE: A HYBRID EXPLAINABLE CYBERSECURITY INVESTIGATION ASSISTANT

AFNAN ALI

BOOTCAMP AI

GIKI

Topi, Pakistan

afnan9075@gmail.com

Abstract—Security analysts frequently combine structured indicators, contextual judgement, and manual reporting when investigating suspicious activity. General-purpose large language models can assist with interpretation, but an LLM-only workflow may be prompt-dependent and can produce variable or unsupported conclusions. This project presents AI Investigation Engine, a hybrid cybersecurity investigation prototype that separates deterministic security decisions from generative AI reasoning. A rule-based analyzer normalizes and deduplicates evidence, applies weighted security indicators, calculates a reproducible 0-100 risk score, assigns LOW, MEDIUM, or HIGH risk, and selects the strongest supported threat category. Google Gemini is then used as an optional advisory layer to correlate evidence, produce an analyst-style summary, explain the detected context, and suggest investigation steps without modifying the deterministic score. The system includes a FastAPI backend, a browser-based command-center interface, report export, graceful AI fallback, and automated testing. The final implementation passes 14 automated tests covering risk levels, duplicate evidence, natural-language variants, threat classification, and AI success/failure paths. The prototype demonstrates how deterministic explainability and LLM flexibility can be combined in a compact security investigation workflow.

Keywords—cybersecurity, threat investigation, explainable AI, Gemini, FastAPI, deterministic risk scoring, security operations

I. INTRODUCTION

Security investigations require analysts to interpret heterogeneous evidence such as failed authentication attempts, suspicious IP addresses, malware indicators, unusual processes, and abnormal network activity. In a small investigation these indicators can be reviewed manually, but repeated case handling creates a consistency problem: the same evidence may be described differently, analysts may weigh indicators differently, and unstructured notes can make reporting slower. At the same time, modern large language models can summarize and reason about natural-language evidence, but they are not inherently deterministic decision engines.

The motivation for this project is therefore not to build another general-purpose chatbot. The objective is to build a structured investigation workflow in which the numerical security assessment is explainable and reproducible, while generative AI is used where it provides the greatest value: contextual correlation and analyst guidance. This design is consistent with the broader explainability principle that AI-assisted systems should provide meaningful reasons for outputs rather than unexplained conclusions [1].

AI Investigation Engine accepts a user-defined case name and one or more evidence statements. Evidence is normalized and exact duplicates are removed before rule evaluation. Recognized indicators contribute fixed weights to a risk score. The resulting score maps to LOW, MEDIUM, or HIGH risk and is accompanied by findings, a threat classification, an attack hypothesis, and recommended response actions. After the deterministic result exists, an optional Gemini reasoning layer receives the evidence and already-calculated outcome. The prompt explicitly prevents the model from recalculating the score or risk level.

The resulting prototype demonstrates a practical hybrid architecture: deterministic logic provides the security decision foundation, while an LLM provides flexible natural-language reasoning. If the Gemini API key is absent or an API call fails, the deterministic result is still returned. This makes AI an enhancement rather than a single point of failure.

II. PROBLEM DEFINITION AND SYSTEM DESIGN

A. Problem Definition

A general-purpose AI assistant can analyze pasted security evidence, but the result depends on how the event is described and how the prompt is written. For an investigation tool, three requirements are especially important: consistent risk scoring, explainable findings, and useful interpretation of free-form evidence. A purely rule-based system satisfies the first two requirements but may be brittle when language varies. A purely generative system is flexible but makes the security decision dependent on a non-deterministic model. The project addresses this trade-off by combining both approaches.

B. Hybrid Architecture

The implemented pipeline is shown in Fig. 1. The browser interface sends a case name and evidence list to a FastAPI endpoint. FastAPI was selected because it provides a lightweight Python API framework with type-oriented request handling and JSON responses [5]. The InvestigationEngine first calls the deterministic Analyzer. Only after the deterministic result is complete does the AIReasoner run.

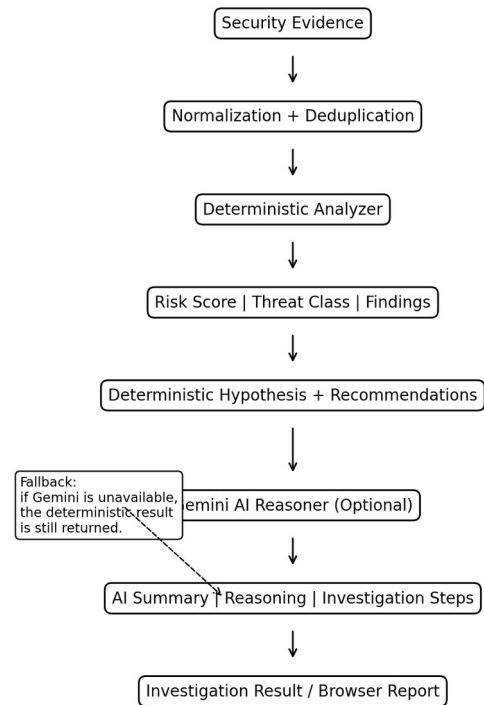


FIGURE 1: Hybrid investigation architecture.

C. Deterministic Risk Analysis

The analyzer uses phrase-based indicator groups with fixed weights. Credential indicators include authentication failures (+30), suspicious or unfamiliar IP context (+40), after-hours access (+20), and brute-force or credential-stuffing language (+50). Malware-related indicators include malware or locked/encrypted-file language (+40) and suspicious or unauthorized processes (+30). Network indicators include outbound communication (+30), large transfer or exfiltration language (+30), and unknown external destinations (+30). The final score is capped at 100.

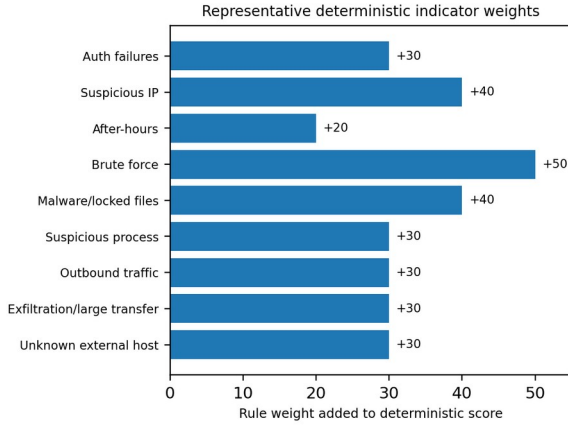


FIGURE 2: Representative deterministic indicator weights.

Risk levels are derived from the deterministic score: scores below 40 are LOW, scores from 40 to 69 are MEDIUM, and scores of 70 or greater are HIGH. Threat classification is calculated separately using weighted category evidence. For mixed incidents, the category with the highest accumulated weight is selected. This prevents an arbitrary if/else ordering from overriding stronger evidence.

Normalization replaces hyphens and underscores, converts text to a consistent form, and removes duplicate evidence before scoring. The vocabulary was intentionally expanded during evaluation to handle ordinary phrases such as 'authentication failures', 'strange IP', 'files became locked', 'cannot access their data', 'data breach', and British/American spelling variants. This improves usability without changing the deterministic architecture.

D. Gemini Reasoning Layer

The optional AIReasoner uses the Google GenAI SDK with Gemini 3.5 Flash-Lite. Google's documentation describes Flash-Lite as a low-latency, cost-effective model intended for high-throughput workloads [3]. The project requests JSON structured output validated by a Pydantic schema containing three fields: summary, reasoning, and investigation_steps. Google documents Pydantic-backed structured output as a supported pattern for Gemini APIs [4].

The Gemini prompt receives the case name, evidence, deterministic risk level, deterministic numerical score, threat classification, and rule-based findings. It is explicitly instructed not to change or recalculate the score or risk level and not to claim that remediation actions were executed. Consequently, Gemini contributes advisory reasoning rather than authoritative detection.

The AI layer is wrapped in a failure boundary. Missing credentials, malformed output, network errors, quota issues, or other exceptions return a fallback object with ai_available set to false. The investigation still contains the deterministic score, threat, findings, hypothesis, and recommendations.

III. IMPLEMENTATION AND EXPERIMENTAL EVALUATION

A. System Implementation

The backend is implemented in Python using FastAPI, Pydantic models, python-dotenv configuration, and the Google GenAI SDK. The frontend uses plain HTML, CSS, and JavaScript and presents the application as a cinematic cyber-operations console. Users can load controlled demonstration incidents or enter their own case and evidence. The results interface displays risk level, risk score, threat class, hypothesis, findings, deterministic recommendations, AI status, AI summary, reasoning, suggested investigation steps, and session history. Reports can be exported as text.

The interface was deliberately designed as a workflow rather than a single text box. A startup screen opens into an operations overview, followed by evidence intake, a visible analysis sequence, a risk visualization, and a separate AI reasoning panel. The visual separation mirrors the backend separation: deterministic findings and the numerical score are shown independently from the Gemini intelligence section. This makes it easier for an analyst or evaluator

to see which outputs come from reproducible rules and which outputs are generative.

B. Evaluation Method

Evaluation focuses on correctness, consistency, and failure handling rather than claiming machine-learning classification accuracy. The final automated suite contains 14 tests. The tests are run using pytest, a Python test framework designed for readable automated testing [7]. The suite verifies low, medium, and high risk behavior; duplicate-evidence protection; hyphen normalization; natural-language credential, malware, and network scenarios; strongest-category selection for mixed evidence; British spelling support; informal evidence handling; Gemini fallback without an API key; mocked structured Gemini success; and safe fallback on simulated API failure.

AI-related tests are deliberately offline-safe. The test configuration removes the real GEMINI_API_KEY, and successful Gemini behavior is exercised with mocks. This prevents the automated suite from depending on API quota or external network availability and makes the test result reproducible.

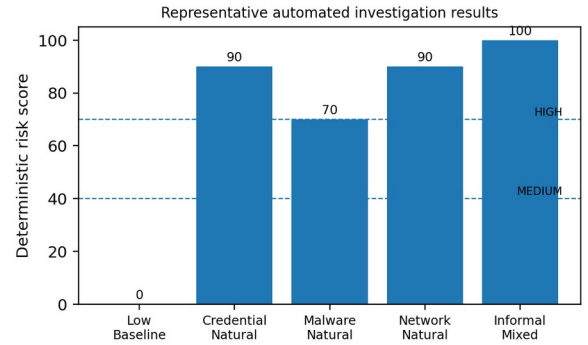


FIGURE 3: Representative deterministic scores from automated scenarios.

C. Representative Results

Representative test results are summarized in Table I. A natural-language credential case containing authentication failures, an unrecognized IP address, and after-hours access produces a score of 90/100 and Credential Attack classification. A natural-language malware case containing malicious software and a suspicious process produces 70/100 and Malware Activity. A network case containing outbound traffic, data exfiltration, and an unknown external server produces 90/100 and Network Anomaly. An informal mixed case describing locked files, inaccessible data, failed logins, and a strange IP address produces a capped HIGH score of 100/100.

Normal login: 0/100, LOW - Suspicious Activity
Credential natural: 90/100, HIGH - Credential Attack
Malware natural: 70/100, HIGH - Malware Activity
Network natural: 90/100, HIGH - Network Anomaly
Informal mixed: 100/100, HIGH - strongest weighted class

TABLE I: Representative deterministic evaluation cases.

The complete test run finishes with 14 passed tests. A single Python 3.14 deprecation warning originates from the installed google-genai library and does not represent a failed project test. The result demonstrates that the final implementation is internally consistent across the tested risk, normalization, natural-language, and AI fallback paths.

D. Explainability and Product Value

The primary value of the prototype is workflow structure, not an attempt to outperform a general-purpose chatbot. An analyst could paste the same evidence into ChatGPT, Claude, or Gemini, but would still need to formulate a prompt, decide how much to trust the response, and manually impose a repeatable risk model. AI Investigation Engine performs the deterministic portion automatically, presents the exact findings that contributed to the decision, and then asks the LLM to explain the already-established context.

This separation also improves failure behavior. If the LLM is unavailable, the core risk score and findings remain usable. If the LLM generates poor reasoning, it cannot silently change the deterministic numerical assessment. The design therefore treats

generative output as analyst assistance rather than as an autonomous security authority.

E. Development Process and Reproducibility

The project was developed using version-control practices with feature branches for engine hardening, AI reasoning, final backend hardening, mission-control user-interface work, and natural-language evidence hardening. Meaningful commits were used to isolate changes such as duplicate-evidence protection, centralized normalization, Gemini integration, custom-evidence improvements, configuration cleanup, frontend redesign, and final natural-language coverage. The completed code was merged into the main branch only after the relevant automated tests passed.

Configuration is separated from source code using a local .env file, while a tracked .env.example documents the required Gemini variables without exposing the API key. The testing design further improves reproducibility by ensuring that pytest does not make uncontrolled live Gemini calls. This means the deterministic and mocked-AI behavior can be re-evaluated without depending on an external quota at test time.

F. Limitations and Security Considerations

The deterministic layer remains phrase-based. Although natural-language coverage was expanded, previously unseen wording may still fail to trigger a rule even when Gemini understands the broader incident. The prototype is therefore a demonstrator rather than a production intrusion-detection system. Future work could introduce a maintained indicator knowledge base, semantic matching, model-based classification, or integration with structured SIEM telemetry while preserving an explainable decision path.

Raw user evidence is included in the LLM prompt. Prompt injection is a known security risk in LLM applications because crafted input can influence model behavior [6]. In this project the impact is reduced by keeping scoring and threat determination outside the LLM boundary; nevertheless, a production version should introduce stronger prompt/data separation, validation, logging, least-privilege integration, and output controls.

Other limitations include browser-session-only history, text rather than PDF report export, no persistent case database, no authentication or multi-user access control, no automated remediation, and no live SOC/SIEM connector. These omissions are intentional for the capstone scope and make clear that the current system is an investigation-assistance prototype rather than a production security platform.

IV. CONCLUSION

AI Investigation Engine demonstrates a hybrid approach to cybersecurity investigation in which deterministic logic and generative AI have deliberately different responsibilities. The deterministic analyzer provides reproducible scoring, findings, threat classification, risk level, attack hypothesis, and response recommendations. Gemini adds contextual correlation, analyst summaries, and suggested investigation steps, while a fallback path preserves the core result when AI is unavailable.

The final implementation supports both predefined demonstrations and user-written evidence, including informal natural-language variations. The automated suite passes 14 tests covering deterministic behavior, evidence normalization, mixed threat handling, and Gemini success/failure paths. The most important contribution is not a claim of autonomous cyber defense; it is the design of a compact, explainable investigation workflow in which the LLM assists rather than controls the security decision.

Future development could add persistent case storage, richer security-event ingestion, SIEM integration, semantic evidence matching, benchmark datasets, analyst feedback loops, stronger LLM security controls, and measured performance comparisons against manual or general-purpose AI workflows. These extensions would allow the prototype to evolve into a more scalable security operations support tool while retaining its deterministic decision boundary.

REFERENCES

- [1] P. J. Phillips, C. A. Hahn, P. C. Fontana, A. N. Yates, K. Greene, D. A. Broniatowski, and M. A. Przybocki, "Four Principles of Explainable Artificial Intelligence," NISTIR 8312, National Institute of Standards and Technology, 2021.
- [2] National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," NIST AI 100-1, 2023.
- [3] Google AI for Developers, "Gemini 3.5 Flash-Lite," Gemini API model documentation, 2026.
- [4] Google AI for Developers, "Structured outputs," Gemini API documentation, 2026.
- [5] FastAPI, "FastAPI Documentation," official project documentation, 2026.
- [6] OWASP GenAI Security Project, "LLM01:2025 Prompt Injection," OWASP Top 10 for Large Language Model Applications, 2025.
- [7] pytest development team, "pytest documentation," official project documentation, 2026.